CODE QUALITY / LOCAL OPERATIONS

SonarQube Guide

One shared server. Repository-owned analysis configuration. Explicit, manually scheduled scans.

Document	Scope
MSC-SQ-001 / 1.2 / EN	Backend, Template, Base and Components
Tool repository	myscoutee-sonarqube
Runtime	Pinned SonarQube Community Build, PostgreSQL and SonarScanner Docker images

Start from the shared repository

The source repository is an explicit argument. Its normal application startup, builds and tests remain independent.

```
cd ../myscoutee-sonarqube
./run.sh ../myscoutee-backend check
./run.sh ../myscoutee-backend up
./run.sh ../myscoutee-backend scan backend
./run.sh ../myscoutee-backend stop
```

Replace the source argument to select another project for analysis. All projects use the same dashboard at localhost:9000. Scans never start another server.

First use on a new machine

Run init once for the shared server. It creates a private database password. Start the instance, change the initial admin password in its UI, and store an analysis token in the existing ignored .state/myscoutee/.env file. Existing migrated credentials are retained; init never overwrites them.

Linux prerequisite: vm.max_map_count must be at least 524288. If needed, explicitly run sudo bash setup-host.sh on the Docker host. The helper preserves higher values and persists the setting.

Configuration and analysis inputs

Keep ownership clear

Location	Contents
Application / config/sonarqube/project.json	Analysis ID and owned module base directories; no server lifecycle settings.
Application / config/sonarqube/	backend, frontend, components and microservices .properties files; project keys and source rules.
Shared tool / server.json	One server identity, Compose project and loopback port for every analysis.
Shared tool / .state/<id>/	Separate bytecode, scanner cache and reports. Only .state/myscoutee/.env owns shared credentials.

Choose the analysis scope

```
./run.sh ../myscoutee-backend scan backend
./run.sh ../myscoutee-backend scan microservices
./run.sh ../myscoutee-backend scan frontend
./run.sh ../myscoutee-components scan components
./run.sh ../myscoutee-base scan backend
```

scan all runs the selected repository's configured modules serially. It is an explicit broad operation; select individual modules when other contributors own frontend or components work. The runner never starts a server or executes a build or test implicitly.

Java bytecode and coverage

Build matching artifacts manually through the application repository's existing build owner. Use the existing Spring Boot JAR for backend and communication assembly JAR for microservices. BACKEND_JAR and MICROSERVICES_JAR select another artifact. Stale inputs stop the scan.

```
BACKEND_COVERAGE_REPORT=server/projects/profile/build/junit/reports/jacoco.xml \
  ./run.sh ../myscoutee-backend scan backend

MICROSERVICES_COVERAGE_REPORT=microservices/build/bootstrap-tests/reports/jacoco.xml \
  ./run.sh ../myscoutee-backend scan microservices
```

Base: explicitly run ./gradlew prepareSonar --parallel from base/server. Paths resolve against the source repository. Coverage is imported only when explicitly supplied. Missing, malformed or stale reports stop the scan; no tests are started to replace them.

Resources, history and findings

Schedule one substantial analysis batch

Fix complete rule families and validate them with focused checks before launching another full scan. The central scan lock covers both applications, artifact preparation and server-side processing, preventing overlapping central scanner runs. Application resource and watcher rules still apply.

Container	Memory limit	CPU limit
SonarQube server	4 GiB	4
PostgreSQL	512 MiB	1
One-shot scanner	4 GiB	6

Keep capacity for the shared server and one scanner together. Limits can be overridden explicitly; CPU quotas do not pin cores. Historical stopped servers retain their volumes without consuming server memory. Scanner Java heap is capped at 1536 MiB; TypeScript analysis uses a 2048 MiB Node heap.

Preserve history; stop deliberately

```
./run.sh ../myscoutee-backend status
./run.sh ../myscoutee-backend logs
./run.sh ../myscoutee-backend stop
./run.sh ../myscoutee-backend up
```

The shared server retains the existing myscoutee-sonarqube Compose name and volumes. The old template database and credentials remain untouched; their history is not automatically merged. stop retains containers and data; down removes the shared server's containers/network while retaining volumes. Automatic restart is disabled.

Interpret the result correctly

A successful scanner upload and server processing confirm that analysis completed. A passing Quality Gate confirms only the gate's configured conditions; it does not mean zero overall findings. Track Security, Reliability, Maintainability, coverage and duplication separately, comparing the same source scope.

Reviewed findings are separate from repaired defects. REVIEWED_FINDINGS.md records the narrow, tested service-only CSRF exception. Other findings remain actionable; neither thresholds nor broad exclusions should be changed to present an incomplete cleanup as complete.

Preparation remains outside the application lifecycle

Sources are mounted read-only. Helm analysis reuses the deployment owner's chart values with isolated placeholder credentials and disposable Git provenance. It does not contact the lab VM or alter source history. Generated artifacts, runtime credentials and scanner caches stay in the shared repository's ignored private state.